DBMS – ADVANCED SQL QUERY SYSTEM

Project Title: ToyWorld – E-Commerce Management System

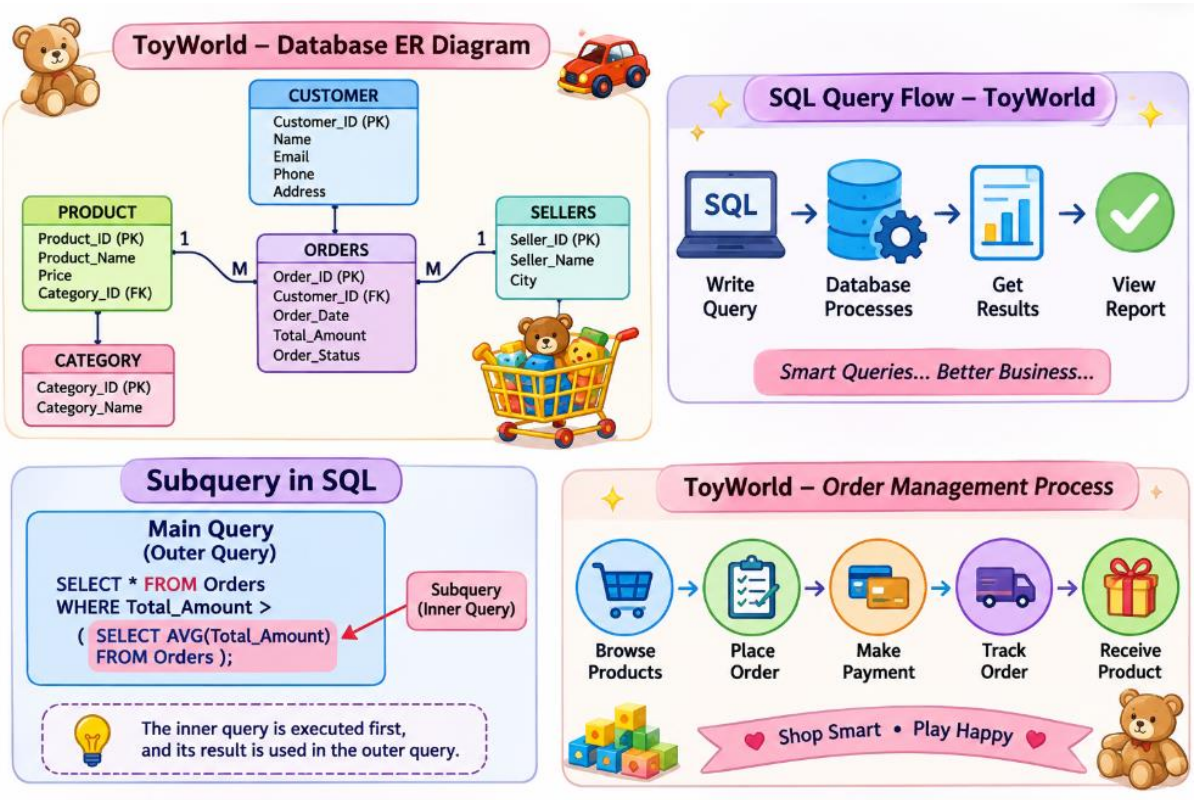
Objectives

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query.

The inner query is executed first, and its result is used by the outer query.



Types of Subqueries

Type	Description
Single-row subquery	Returns one value
Multiple-row subquery	Returns multiple values
Nested subquery	A subquery inside another subquery
Correlated subquery	Inner query depends on outer query

Example

Find orders whose amount is greater than the average order amount.

```
SQL> SELECT ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT
  2  FROM ORDERS
  3  WHERE TOTAL_AMOUNT >
  4  (SELECT AVG(TOTAL_AMOUNT)
  5   FROM ORDERS);
```

```
  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
---------- ----------- ------------
      1001           1         1350
      1003           3         1500
      1007           2         1800
```

Explanation:

The inner query calculates the average order amount.

The outer query displays orders whose total amount is greater than the average amount.

2. Find Products Above Average Price

A subquery can be used to compare each product's price with the average product price.

Assuming the PRODUCT table contains:

- PRODUCT_ID
- PRODUCT_NAME
- PRICE

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM PRODUCT
  3  WHERE PRICE > (SELECT AVG(PRICE) FROM PRODUCT);
```

```
PRODUCT_ID PRODUCT_NAME          PRICE
---------- -------------------- ------
       103 Remote Car             1200
       104 Chess Board             750
```

Explanation:

The inner query calculates the average product price.

The outer query displays products whose price is greater than the calculated average.

2.1 Products Below Average Price

The same concept can be used to find products below the average price.

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM PRODUCT
  3  WHERE PRICE <
  4  (SELECT AVG(PRICE)
  5   FROM PRODUCT);
```

```
PRODUCT_ID PRODUCT_NAME          PRICE
---------- -------------------- ------
       101 ABC Learning Kit        450
       102 Teddy Bear              550
       105 Football                450
       109 Superhero Figure        600
```

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT CUSTOMER_ID,
  2  SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM ORDERS
  4  GROUP BY CUSTOMER_ID;
```

```
CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
          1           2450
          2           2650
          3           2250
          4            650
          5            950
```

Step 2: Find the Maximum Purchase

```
SQL> SELECT MAX(TOTAL_PURCHASE)
  2  FROM
  3  (
  4   SELECT CUSTOMER_ID,
  5   SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  6   FROM ORDERS
  7   GROUP BY CUSTOMER_ID
  8  );
```

```
MAX(TOTAL_PURCHASE)
-------------------
               2650
```

Explanation:

The inner query calculates the total purchase for each customer.

The outer query finds the maximum purchase amount.

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT C.ID, C.NAME,
  2  SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM CUSTOMER C
  4  JOIN ORDERS O
  5  ON C.ID = O.CUSTOMER_ID
  6  GROUP BY C.ID, C.NAME
  7  HAVING SUM(O.TOTAL_AMOUNT) =
  8  (
  9   SELECT MAX(TOTAL_PURCHASE)
 10   FROM
 11   (
 12    SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
 13    FROM ORDERS
 14    GROUP BY CUSTOMER_ID
 15   )
 16  );
```

```
     ID NAME                           TOTAL_PURCHASE
------- ------------------------------ --------------
      2 Khilan                                 2650
```

Explanation:

The query displays the customer ID, customer name and total purchase amount of the customer who made the maximum purchases.

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT C.ID, C.NAME,
  2  O.ORDER_ID, O.TOTAL_AMOUNT
  3  FROM CUSTOMER C
  4  JOIN ORDERS O
  5  ON C.ID = O.CUSTOMER_ID
  6  WHERE O.TOTAL_AMOUNT >
  7  (
  8   SELECT AVG(TOTAL_AMOUNT)
  9   FROM ORDERS
 10  )
 11  ORDER BY O.TOTAL_AMOUNT DESC;
```

```
   ID NAME                           ORDER_ID TOTAL_AMOUNT
------ ------------------------------ ---------- ------------
     2 Khilan                           1007         1800
     3 kaushik                          1003         1500
     1 Ramesh                           1001         1350
```

Purpose:

This query identifies customers who placed orders whose value is greater than the overall average order value.

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated using the PRODUCT and ORDER_ITEM tables.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OI.QUANTITY * OI.UNIT_PRICE) AS TOTAL_SALES
  4  FROM PRODUCT P
  5  JOIN ORDER_ITEM OI
  6  ON P.PRODUCT_ID = OI.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  HAVING SUM(OI.QUANTITY * OI.UNIT_PRICE) >
  9  (
 10   SELECT AVG(PRODUCT_SALES)
 11   FROM
 12   (
 13    SELECT SUM(QUANTITY * UNIT_PRICE) AS PRODUCT_SALES
 14    FROM ORDER_ITEM
 15    GROUP BY PRODUCT_ID
 16   )
 17  )
 18  ORDER BY TOTAL_SALES DESC;
```

```
PRODUCT_ID PRODUCT_NAME         TOTAL_SALES
---------- -------------------- -----------
       103 Remote Car                4150.00
       101 ABC Learning Kit          2400.00
```

Purpose:

This query identifies products whose total sales are greater than the average sales generated by all products.

6. Customer Purchase Report

A detailed customer purchase report can combine multiple tables.

```
SQL> SELECT C.ID, C.NAME,
  2  COUNT(O.ORDER_ID) AS TOTAL_ORDERS,
  3  SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE,
  4  AVG(O.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  5  FROM CUSTOMER C
  6  JOIN ORDERS O
  7  ON C.ID = O.CUSTOMER_ID
  8  GROUP BY C.ID, C.NAME
  9  ORDER BY TOTAL_PURCHASE DESC;
```

ID	NAME	TOTAL_ORDERS	TOTAL_PURCHASE	AVERAGE_ORDER_VALUE
2	Khilan	2	2650	1325
1	Ramesh	2	2450	1225
3	kaushik	2	2250	1125
5	Hardik	1	950	950
4	Chaitali	1	650	650

Report Contains:

- Customer ID
- Customer Name
- Number of Orders
- Total Purchase Amount
- Average Order Value

7. Advanced Product Sales Report

This report combines PRODUCT and ORDER_ITEM tables to analyse product-wise sales performance.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         COUNT(DISTINCT OI.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         SUM(OI.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  5         SUM(OI.QUANTITY * OI.UNIT_PRICE) AS TOTAL_SALES,
  6         AVG(OI.UNIT_PRICE) AS AVERAGE_PRICE
  7  FROM PRODUCT P
  8  JOIN ORDER_ITEM OI
  9  ON P.PRODUCT_ID = OI.PRODUCT_ID
 10  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 11  ORDER BY TOTAL_SALES DESC;
```

PRODUCT_ID	PRODUCT_NAME	NUMBER_OF_ORDERS	TOTAL_QUANTITY_SOLD	TOTAL_SALES	AVERAGE_PRICE
103	Remote Car	2	5	4150.00	800
101	ABC Learning Kit	2	5	2400.00	450
104	Chess Board	1	1	1500.00	1500
109	Superhero Figure	1	2	1100.00	550
105	Football	1	1	650.00	650

Purpose:

This report helps the ToyWorld business identify:

- Number of orders for each product.
- Total quantity sold.
- Total sales amount.
- Average selling price.

- Products generating higher sales.

8. Category-wise Advanced Sales Report

Category-wise sales can be generated by combining CATEGORY, PRODUCT and ORDER_DETAILS tables, if these tables are available.

```
SQL> SELECT C.CATEGORY_ID,
  2         C.CATEGORY_NAME,
  3         COUNT(DISTINCT OI.ORDER_ID) AS TOTAL_ORDERS,
  4         SUM(OI.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  5         SUM(OI.QUANTITY * OI.UNIT_PRICE) AS TOTAL_SALES,
  6         AVG(OI.UNIT_PRICE) AS AVERAGE_PRICE
  7  FROM CATEGORY C
  8  JOIN PRODUCT P
  9  ON C.CATEGORY_ID = P.CATEGORY_ID
 10  JOIN ORDER_ITEM OI
 11  ON P.PRODUCT_ID = OI.PRODUCT_ID
 12  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 13  ORDER BY TOTAL_SALES DESC;
```

CATEGORY_ID	CATEGORY_NAME	TOTAL_ORDERS	TOTAL_QUANTITY_SOLD	TOTAL_SALES	AVERAGE_PRICE
3	Remote Control Toys	2	5	4150.00	800.00
1	Educational Toys	2	5	2400.00	450.00
4	Board Games	1	1	1500.00	1500.00
9	Action Figures	1	2	1100.00	550.00
5	Outdoor Toys	1	1	650.00	650.00

Purpose

This report helps ToyWorld identify:

- Category-wise total orders.
- Total quantity sold.
- Total sales amount.
- Average selling price.
- Best-selling categories.

Conclusion

The ToyWorld E-Commerce Management System uses advanced SQL queries to manage customer purchases, orders and seller performance. Subqueries, joins and aggregate functions help generate useful business reports and improve database management.